

PHOENIX – TEAVS–ADCRS Integration

Readiness Analysis

Project: PHOENIX

Stream: Cybersecurity

Trimester: Trimester 2, 2026

Prepared by: Abdul Rehman

Role: Cybersecurity Stream Leader

Technical Task: Implement TEAVS–ADCRS Integration

1. Purpose

The purpose of this document is to review the existing work related to the integration between TEAVS (Trusted Emergency Alert Verification System) and ADCRS (AI-Driven Cyber Risk Scoring System).

The TEAVS–ADCRS integration is one of the current cybersecurity priorities for PHOENIX. However, the Sprint 1 plan identifies that full integration depends on the completion of required backend components.

Therefore, before starting implementation, I reviewed the existing security designs, threat-detection work and risk-scoring functionality to understand what is already available and what is still required.

2. Existing TEAVS Security Design

Existing PHOENIX documentation shows that several security controls have already been designed for TEAVS.

These include:

- JWT authentication
- Role-Based Access Control (RBAC)
- Digital signatures
- Data integrity verification
- Input validation
- Rate limiting
- Secure API communication
- Secure error handling

These controls provide a security foundation that can be considered when TEAVS is integrated with other PHOENIX components.

3. JWT Authentication and Access Control

The existing TEAVS authentication design uses JWT tokens to control access to protected API functionality.

The proposed flow is:

User Login

↓

Credentials Verified

↓

JWT Generated

↓

JWT Sent with API Requests

↓

Token Validated

↓

User Role Checked

↓

Access Granted / Denied

The JWT design includes user roles that can be used for Role-Based Access Control.

For the TEAVS–ADCRS integration, the current implementation of authentication and authorisation should be reviewed to determine whether existing controls can also protect relevant integration endpoints.

4. Digital Signature Design

Existing TEAVS documentation also proposes digital signatures to protect the integrity and authenticity of information.

The proposed process is:



This is relevant to TEAVS–ADCRS because information exchanged between the systems may require verification before it is trusted and processed.

The actual implementation of this mechanism still needs to be confirmed.

5. Existing Threat-Detection Prototype

An existing Python threat-detection prototype was also reviewed.

The prototype contains rule-based detection for several cybersecurity threats, including:

- Login attacks
- API abuse
- Data breaches
- Phishing and scams
- Malware and ransomware
- DDoS attacks
- IoT attacks
- Misinformation and deepfakes
- Alert-system compromise
- Fraudulent claims

The functions return structured information containing:

```
{  
  "threat": "API Abuse",  
  "severity": "High",  
  "action": "rate_limit"  
}
```

This provides a simple structure containing the detected threat, its severity and a recommended response.

A particularly relevant function for TEAVS checks whether an alert sender is authorised, whether the alert has approval and whether authentication is valid.

If these checks fail, the prototype identifies a critical **Alert System Compromise** and recommends disabling alert dispatch.

This functionality may be useful when defining how TEAVS security events interact with other PHOENIX components.

6. Existing AI Risk-Scoring Functionality

I also reviewed the existing PHOENIX core-model integration script.

The script prepares backend input for a trained XGBoost model and returns a structured risk result.

The model accepts information including:

- URL
- Text
- Timestamp
- Hazard type
- Hazard severity
- Hazard location
- Hazard status
- Alert level
- Data source

The output contains:

```
{  
  "risk_score": 0.87,  
  "confidence_score": 0.76,  
  "predicted_class": 3,  
  "risk_level": "Critical"  
}
```

The script also performs input validation before running the model.

This existing risk-scoring functionality is relevant to the ADCRS side of the planned integration. However, its exact relationship with the complete ADCRS architecture still needs to be confirmed.

7. Current Integration Understanding

Based on the existing material reviewed so far, the high-level integration may involve TEAVS providing verified alert, threat or hazard information that can then be processed through PHOENIX backend services and risk-scoring functionality.

A possible high-level flow is:

TEAVS

↓

Verified Alert / Threat Information

↓

PHOENIX Backend

↓

Risk-Scoring / ADCRS Functionality

↓

Risk Score + Confidence + Risk Level

↓

PHOENIX Backend

↓

Relevant PHOENIX Interface

This is currently an **initial integration understanding**, not a confirmed final architecture.

The final flow needs to be confirmed with the Backend and AI/ML teams before implementation.

8. Security Requirements

Based on the existing security work, the following areas should be considered during implementation:

Authentication

Only authenticated users or services should be able to access protected functionality.

Authorisation

Roles and permissions should be checked before access to protected operations is allowed.

Input Validation

Data received by the integration should be validated before being processed.

Data Integrity and Authenticity

Where required, digital signatures can be used to verify that information has not been modified and comes from a trusted source.

Secure Communication

Communication between components should use secure communication channels.

Rate Limiting

Relevant exposed APIs should be protected against excessive or abusive requests.

Error Handling

Errors should be handled without exposing sensitive internal information.

Logging

Important authentication, integration and security events should be recorded to support monitoring and investigation.

9. Current Dependency / Blocker

The current Sprint 1 cybersecurity plan identifies the following issue:

TEAVS and ADCRS integration depends on the completion of backend components.

This means full end-to-end implementation cannot be completed until the required backend functionality and interfaces are available.

Before implementation, the following still need to be confirmed:

- Required backend service(s)
- Available API endpoints
- TEAVS request format
- ADCRS input requirements
- ADCRS response format
- Authentication between components
- How the risk-scoring model is called
- Where integration results are stored
- How results are returned to TEAVS or other PHOENIX interfaces
- Logging and error-handling requirements

10. Work That Can Continue During the Blocker

Although full implementation currently depends on backend development, cybersecurity work can continue.

The following activities can be completed in preparation:

- Review existing TEAVS security documentation
- Review existing threat-detection functionality
- Review the current risk-scoring model interface
- Identify possible data exchanged between components
- Identify security requirements
- Identify trust boundaries
- Prepare threat-modelling scenarios
- Prepare integration test cases
- Prepare security test cases
- Coordinate technical requirements with Backend and AI/ML teams

This preparation will reduce integration issues once the required backend components become available.

11. Next Steps

The next stage of this work will be to:

1. Confirm the TEAVS–ADCRS architecture with the relevant PHOENIX teams.
2. Identify the exact backend components required for integration.
3. Confirm the request and response data formats.
4. Confirm how the existing AI risk-scoring functionality relates to ADCRS.
5. Create a detailed data-flow diagram.
6. Identify trust boundaries and perform threat modelling.
7. Define integration and security test cases.
8. Begin implementation when the required backend components become available.
9. Perform functional and security testing after integration.
10. Document findings, fixes and retesting results.

12. Conclusion

The Week 4 review provided a clearer understanding of the existing security and technical components that may support TEAVS–ADCRS integration.

TEAVS already has security designs covering authentication, access control, digital signatures and secure communication. Existing PHOENIX work also includes rule-based threat detection and an AI model integration that produces structured risk-scoring results.

The main current limitation is the dependency on unfinished or unconfirmed backend components.

The next priority is therefore to confirm the backend interfaces and final data flow before moving from integration analysis into implementation and security testing.